

Cumplimiento del paper Harness Handbook

Verificación contra arXiv:2607.13285v1 (Wang et al.,
Tencent HY LLM Frontier / Indiana University, Jul 2026)
— análisis de gaps y plan de implementación —

Fecha	2026-07-21
Objeto	Harness jcode v100-clean (proyecto-01.zip)
Paper de referencia	Harness Handbook: Making Evolving Agent Harnesses Readable, Navigable, and Editable
Autores paper	Ruhan Wang, Yucheng Shi, Zongxia Li, et al. (Tencent / IU / UMD / UGA / NUS)
Documento complementario	Auditoría técnica del harness (parte 1)
Propósito	Identificar gaps contra el paper y proponer patches de implementación
Severidades	Critical / High / Medium / Low

Índice

1. Resumen ejecutivo	5
1.1 Cumplimiento por pilar del paper	5
2. Requisitos formales del paper	7
2.1 Concepto 1 — Harness Handbook (§ 3.1)	7
2.2 Concepto 2 — BGPD (§ 3.3.1)	7
2.3 Concepto 3 — Construction pipeline (§ 3.2)	8
2.4 Concepto 4 — Resync automático (§ 3.3.3, Appendix B.4)	8
3. Matriz de cumplimiento detallada	10
4. Lo que SÍ cumple el harness	13
4.1 Mappings paper ↔ harness	13
4.2 Conclusión de la sección	14
5. Gap 1 — Construction Pipeline ausente	15
5.1 Lo que el paper exige	15
5.2 Lo que el harness tiene hoy	15
5.3 Impacto de no tener pipeline	15
6. Gap 2 — Resync automático ausente	17
6.1 Lo que el paper exige (Appendix B.4)	17
6.2 Lo que el harness tiene hoy	17
7. Gap 3 — Behavior-Implementation Alignment	19
7.1 Lo que el paper exige	19
7.2 Lo que el harness tiene hoy	19
7.3 Mecanismo de freeze que falta	20
8. Gap 4 — BGPD sin source verification	21
8.1 Lo que el paper exige (§ 3.3.1, Appendix B.1 paso 4)	21
8.2 Lo que el harness tiene hoy	21
9. Gap 5 — Program Graph ausente	23
9.1 Lo que el paper exige (Phase I, § 3.2)	23
9.2 Métricas del paper (Table 2)	23
10. Gap 6 — Edit Planning estructurado	25
10.1 Lo que el paper exige (§ 3.3.2, Appendix B.2)	25

10.2 Lo que el harness tiene hoy	25
11. Gap 7 — Leaf mode ausente	26
11.1 Criterio del paper para elegir g (§ 3.2)	26
11.2 Recomendación para el harness jcode	26
12. Gap 8 — SKILL.md manifest canonical	27
12.1 SKILL.md del paper (Figure 6)	27
13. Blueprint de implementación por fases	28
13.1 Timeline resumido	28
13.2 Priorización P0/P1/P2	29
14. Patch 1 — handbook_builder.py (Phase I)	30
14.1 Código — jcode/lib/handbook_builder.py	30
14.2 Extensión a otros lenguajes	34
15. Patch 2 — Phase II con LLM	35
15.1 Código — jcode/lib/handbook_phase2.py	35
16. Patch 3 — Phase III synthesis	39
16.1 Código — jcode/lib/handbook_phase3.py	39
17. Patch 4 — Resync hook post-commit	44
17.1 Código — jcode/lib/handbook_resync.py	44
17.2 Modificación de turnend.sh	48
18. Patch 5 — BGPD con source verification	49
18.1 Código — jcode/lib/handbook_verify.py	49
18.2 Modificación de skill orient/SKILL.md	52
19. Patch 6 — SKILL.md manifest canonical	53
19.1 Código — jcode/skills/handbook/SKILL.md	53
20. Patch 7 — Edit Planning con Γ declarations	55
20.1 Template OP.md extendido	55
21. Roadmap y dependencias	57
21.1 Timeline visual	57
22. Conclusión y recomendación	58
22.1 ¿Vale la pena la inversión?	58
22.2 Recomendación final	58

1. Resumen ejecutivo

Este documento complementa la auditoría técnica del harness jcode v100-clean evaluando su cumplimiento contra el paper **Harness Handbook: Making Evolving Agent Harnesses Readable, Navigable, and Editable** (Wang et al., arXiv:2607.13285v1, 14 julio 2026, Tencent HY LLM Frontier / Indiana University). El paper formaliza tres pilares que un harness de agentes moderno debería implementar: (1) la representación Harness Handbook con un document tree L1-L3 y una vista de state-registers Z; (2) el workflow Behavior-Guided Progressive Disclosure (BGPD) que guía al agente de coarse a fine verificando contra el repo; y (3) un construction pipeline automático en 3 fases (Static Fact Extraction → Behavioral Organization → Hierarchical Synthesis) con resync tras cada diff no vacío.

Veredicto: el harness jcode NO cumple el paper en sus 3 pilares centrales. Cumple parcialmente en **filosofía** (es behavior-centric, declara progressive disclosure, separa estados de comportamientos) pero le falta **todo el aparato técnico** que el paper exige: no hay construction pipeline, no hay resync automático, no hay verificación de locators contra el repo, no hay program graph, no hay edit planning estructurado con action declarations. Los archivos BEHAVIOR-INDEX.md y STATE-REGISTERS.md del harness son plantillas vacías que se llenan manualmente y se desincronizan al primer commit, justo el problema que el paper resuelve.

La buena noticia es que el harness provee el **andamiaje correcto** para adoptar el paper: BEHAVIOR-INDEX.md es un esqueleto válido de L2 stage cards, STATE-REGISTERS.md es un esqueleto válido de la view Z, la skill orient/ ya declara una política BGPD de 4 niveles, y PRINCIPLES.md ya declara progressive disclosure. Lo que falta es **automatizar** la generación y mantenimiento de estos artefactos. La sección 14 propone un blueprint de 6 fases (~3-4 semanas de implementación) con patches de código listos para aplicar en las secciones 15-21.

1.1 Cumplimiento por pilar del paper

#	Pilar del paper	Cumpli miento	Hallazgo	Gap ref
P 1	Harness Handbook representation (D + Z)	No cumple	El harness tiene BEHAVIOR-INDEX.md (esqueleto L2) y STATE-REGISTERS.md (esqueleto Z) pero son plantillas vacías, sin L1 overview ni L3 entries con locators.	Gap 1
P 2	BGPD workflow (coarse-to-fine + verify)	Parcial	skill orient/ declara la política L1→L2→Z→L3 pero NUNCA verifica los paths contra el filesystem. Falta el paso de source verification.	Gap 4
P 3	Construction pipeline (Phase I/II/III)	No cumple	No existe. Sin static analysis, sin Propose-Review-Mapping, sin Hierarchical Synthesis. Todo se llena manualmente.	Gap 1, 5

#	Pilar del paper	Cumplimiento	Hallazgo	Gap ref
P 4	Resync automático tras diff	No cumple	No existe. turnend.sh hace auditoría post-commit pero NUNCA toca BEHAVIOR-INDEX ni STATE-REGISTERS. El handbook se desincroniza al primer refactor.	Gap 2
P 5	Behavior-Implementation Alignment	No cumple	Nada verifica que los paths en BEHAVIOR-INDEX existan. Un rename deja el handbook mintiendo sin detectarse.	Gap 3
P 6	Edit Planning con Gamma declarations	Parcial	LOOPS.md define fixed_check pero no produce (will_modify, will_add, will_remove) en function granularity para alimentar Resync.	Gap 6
P 7	Leaf mode $g \in \{\text{function}, \text{file}\}$	No cumple	El harness no decide si la leaf es función o archivo. BEHAVIOR-INDEX usa "archivos" informalmente.	Gap 7
P 8	SKILL.md manifest canonical	Parcial	Las skills del harness existen pero no siguen el formato Figure 6 del paper (references/overview.md, index.md, registers.md, stages/.md).	Gap 8

Tabla 1. Cumplimiento por pilar del paper Harness Handbook.

2. Requisitos formales del paper

El paper define formalmente cuatro conceptos centrales. Esta sección los extrae con la notación original para que el lector pueda contrastar contra la implementación del harness. Las referencias a secciones del paper (§ 3.1, § 3.2, § 3.3) y a los algoritmos (Algorithm 1, 2) son del manuscrito arXiv:2607.13285v1.

2.1 Concepto 1 — Harness Handbook (§ 3.1)

El **Handbook** es una representación operacional que reorganiza el conocimiento del repositorio alrededor de **comportamientos runtime** en vez de archivos. Tiene dos partes: un **document tree D** jerárquico L1–L3 y una **vista complementaria Z** de state-registers.

Document tree D (L1–L3): El lector empieza en L1 (system overview: arquitectura, execution model, stages mayores, flujo de datos global), baja a L2 (component overview: responsibilities, inputs/outputs, dependencias, estado local de un stage seleccionado), y finalmente llega a L3 (unit deep dive: entradas source-grounded con locators verificables).

Vista Z (state-registers): Registra relaciones de estado que cruzan stage boundaries. Para cada register, lista todos los writers y readers. Esto permite encontrar stages que son mutuamente dependientes aunque estructuralmente distantes (un valor escrito en stage-3.2 y consumido en stage-7.1).

Dos reglas de mantenimiento: (1) Progressive disclosure — el lector avanza de L1 a L3 solo cuando la tarea lo requiere; (2) Behavior-implementation alignment — cada L3 locator activo debe resolver al repo actual; si no puede revalidarse, la entrada se marca **frozen** y se excluye de localization hasta que se refresque.

2.2 Concepto 2 — BGPD (§ 3.3.1)

Behavior-Guided Progressive Disclosure es el workflow que localiza el comportamiento solicitado mediante navegación coarse-to-fine del handbook, seguida de verificación contra el repo. Tiene 4 pasos:

Pa so	Nombre	Descripción formal
1	Stage selection	Navega L1 y L2 de D para identificar stages directamente relevantes al request q. Sigue Z para incluir stages acoplados vía shared state.
2	Entry selection	Dentro de los stages seleccionados, abre las stage pages y elige los L3 entries más relevantes. Cada entry expone un summary y un source locator.
3	Call-relation expansion	Expande el candidate set a lo largo de call relations en el program graph G. En function-as-leaf sigue la function-call graph; en file-as-leaf la induced file-call graph. External boundaries dan contexto pero nunca se retornan como edit sites.
4	Source verification	Abre el repo R, resuelve cada locator candidato, retiene solo los sites que siguen siendo relevantes a q. Produce el evidence set E_bq con file path, anchor opcional, y excerpt actual.

Tabla 2. Los 4 pasos de BGPD según § 3.3.1 y Appendix B.1.

2.3 Concepto 3 — Construction pipeline (§ 3.2)

El **leaf mode** $g \in \{\text{function}, \text{file}\}$ es fijo para el lifetime del handbook. En **function-as-leaf**, cada L3 entry cubre una función entera o regiones contiguas; se usa cuando hay un seed skeleton S_0 confiable y el budget permite granularidad función. En **file-as-leaf**, cada L3 entry es un archivo; se usa cuando no hay seed o function-level excedería el budget. Una vez elegido g , la construcción procede en 3 fases:

Fase	Nombre	Descripción formal
Phase I	Static Fact Extraction	Adapters language-specific parsean el repo y extraen: funciones, boundaries externos, source locations, signatures, call edges. El program graph G mantiene solo calls que resuelven a función interna o boundary nombrado. Llamas no resueltas se loguean, no se asignan a targets adivinados. Determinístico, cero llamadas LLM.
Phase II	Behavioral Organization	Organiza source units en el execution-stage skeleton S . En function-as-leaf: usa source + call-graph context para proponer function-to-stage assignments, refinadas vía iterative review. En file-as-leaf: describe cada file como card, combina con G para inferir S , organiza files por stage. LLM-assisted con Propose-Review-Mapping loop.
Phase III	Hierarchical Synthesis	Convierte (S, U_g) en el document tree D y la vista Z . Cada L3 entry se linkea a una source location statically identificada y se valida contra el repo. Rendera la vista reader-facing V y empaqueta el estado de resync K_g . LLM para descripciones; paths/locators son facts.

Tabla 3. Las 3 fases del construction pipeline (§ 3.2).

2.4 Concepto 4 — Resync automático (§ 3.3.3, Appendix B.4)

Cada diff no vacío Δ dispara $\text{Resync_g}(H, R, R', \Delta, \Gamma)$ que actualiza solo las partes afectadas del handbook cuando es posible. Tiene 3 pasos:

(1) **Version alignment:** reparses R' , construye el program graph actualizado G' , compara con G usando Δ para identificar added/removed/modified units. En function-as-leaf, funciones se matchean por body fingerprints que ignoran line numbers (un rename se detecta matcheando el body bajo la signature line). En file-as-leaf, file-set differences + content hashes.

(2) **Scoped update:** si el stage skeleton S sigue siendo válido, solo se actualizan las entries afectadas y su enclosing structure (L3 entries $\rightarrow$ L2/L1 ancestors $\rightarrow$ state registers dependientes). Output generation se reusa del cache B para contenido no cambiado. Si S ya no acomoda el cambio, se rerunnea Algorithm 2 completo (full rebuild).

(3) **Conservative handling:** source que no se puede parsear o clasificar no se adivina. L3 entries cuyo locator o hash no se puede revalidar se marcan frozen y se excluyen de localization. Functions cambiadas que no se pueden clasificar quedan registradas como

unmapped. **Model calls limitadas a 4 categorías semánticas:** classification, file assignment, within-stage organization, description revision. Todo lo demás es determinístico.

3. Matriz de cumplimiento detallada

Cada requisito formal del paper cruzado con el estado actual del harness. La columna "Implementación paper" describe lo que el paper exige; "Estado harness" describe lo que el harness tiene hoy; "Cumplimiento" es el veredicto; "Gap ref" apunta a la sección donde se desarrolla el gap y se propone patch.

ID	Requisito paper	Implementación paper	Estado harness	Cumplimiento	Gap ref
R-01	Document tree D con niveles L1, L2, L3	Generación automática desde (S, U_g) en Phase III	No existe. BEHAVIOR-INDEX.md es plantilla con placeholders B-XXX.	No cumple	Gap 1
R-02	Vista Z de state-registers con writers/readers	Generada en Phase III desde declarations en S (function-mode) o propuesta desde L1 summaries (file-mode)	STATE-REGISTERS.md es plantilla con placeholders <tabla>.<columna>.	No cumple	Gap 1
R-03	Leaf mode $g \in \{\text{function}, \text{file}\}$ fijo	Decidido al construir el handbook, no cambia	El harness no tiene concepto de leaf mode.	No cumple	Gap 7
R-04	L3 entries con source locator verificable	Cada L3 entry tiene file path + anchor opcional, validado contra R	BEHAVIOR-INDEX lista "archivos" sin locators ni anchors ni verificación.	No cumple	Gap 3
R-05	Program graph G con calls + state read/write	Phase I: AST parse, function nodes, boundary nodes, resolved call edges, state accesses	No existe. El harness usa grep/rg ad-hoc.	No cumple	Gap 5
R-06	Stage skeleton S (function-mode: desde S ₀ seed)	Phase II function-mode: Propose-Review-Mapping loop con LLM clasificando functions en stages	No existe stage skeleton. PRINCIPLES PARTE II define task_class pero no execution stages.	No cumple	Gap 1
R-07	Stage skeleton S (file-mode: inferido)	Phase II file-mode: file cards + G → infer S, organizar files por stage	No existe. Sin cards ni organización automática.	No cumple	Gap 1

ID	Requisito paper	Implementación paper	Estado harness	Cumplimiento	Gap ref
R-08	BGPD paso 1: stage selection via L1+L2+Z	Navega D y Z para identificar stages relevantes + acoplados por shared state	skill orient/ Fase 2-B paso 1 declara esto pero sin automatización.	Parcial	Gap 4
R-09	BGPD paso 2: entry selection dentro de stages	Abre stage pages, elige L3 entries relevantes	skill orient/ Fase 2-B paso 1 lo declara (L1 BEHAVIOR-INDEX).	Parcial	Gap 4
R-10	BGPD paso 3: call-relation expansion	Expande via function-call graph (function-mode) o file-call graph (file-mode)	No existe. Sin program graph G no hay expansion.	No cumple	Gap 5
R-11	BGPD paso 4: source verification	Abre R, resuelve locators, descarta los que no matchean	skill orient/ NUNCA verifica paths contra filesystem.	No cumple	Gap 4
R-12	Edit plan P con old/new pairs byte-exact	Planner convierte E_bq en bloques edit con old_string verbatim + new_string	LOOPS.md define fixed_check pero no edit blocks estructurados.	Parcial	Gap 6
R-13	Action declarations $\Gamma = (\Gamma_{\text{modify}}, \Gamma_{\text{add}}, \Gamma_{\text{remove}})$	Por edit: target file, anchor, action type. Rename = remove + add	No existe. Sin Γ no se puede auditar execution contra plan ni alimentar Resync.	No cumple	Gap 6
R-14	Executor separado del planner	Applies P mecánicamente, no re-reads, verifica syntax gate	worker-ejecutor skill cumple parcialmente. No hay separation estricta.	Parcial	—
R-15	Resync tras cada diff no vacío	Resync_g(H, R, R', Δ , Γ): version align + scoped update + conservative handling	No existe. turnend.sh audita pero no actualiza BEHAVIOR-INDEX/STATE-REGISTERS.	No cumple	Gap 2
R-16	Frozen entries para locators no revalidables	L3 entry sin locator válido → frozen, excluida de localization	No existe mecanismo de freeze.	No cumple	Gap 3
R-17	Coverage record Y para units no clasificables	Functions/files no mapeados quedan visibles en Y, no se asignan por guesswork	No existe. Las plantillas no tienen sección "unmapped".	No cumple	Gap 1

I D	Requisito paper	Implementación paper	Estado harness	Cumpl imient o	Gap ref
R - 1 8	Cache B para outputs de generación reusables	Contenido no cambiado se reusa del cache en resync	No existe cache de handbook.	No cu mple	Gap 2
R - 1 9	SKILL.md manifest canonical (Figure 6)	references/overvi ew.md, index.md, registers.md, stages/<id>.md	Skills orient/ y arquitecto-proyecto/ no siguen este layout.	Parcial	Gap 8
R - 2 0	Model calls limitadas a 4 categorías semánticas	classification, file assignment, within-stage organization, description revision	N/A — no hay pipeline. Pero la restricción aplica cuando se implemente.	N/A	—

Tabla 4. Matriz detallada de cumplimiento (20 requisitos del paper).

4. Lo que SÍ cumple el harness

A pesar del veredicto negativo global, el harness jcode tiene varios aciertos que mapean directamente a conceptos del paper. Esta sección identifica esos mappings exactos para que la implementación de los patches (secciones 15-21) pueda reutilizar lo existente en vez de empezar desde cero.

4.1 Mappings paper ↔ harness

Componente harness	Concepto paper	Mapeo	Falta para cumplir
BEHAVIOR-INDEX.md	L2 stage cards + L3 entries esqueleto	El archivo tiene la estructura correcta: B-XXX entries con "Archivos", "Estados", "Reglas", "Loop típico", "Tests críticos". Es exactamente el formato L2/L3 del paper.	Falta: locators verificables (file:start-end), L1 overview generado, automation.
STATE-REGISTERS.md	Vista Z (state-register view)	La plantilla tiene la estructura correcta: tabla por campo con Writes, Reads, Invariantes. Es la vista Z del paper.	Falta: auto-extraction de writers/readers desde el código, cross-stage tracing.
skill orient/SKILL.md	BGPD policy	Fase 2-B paso 1 declara BGPD: "L1: Leer BEHAVIOR-INDEX → L2: archivos y reglas → Z: STATE-REGISTERS → L3: grep para confirmar existencia". Es exactamente el workflow del paper.	Falta: source verification real (el paso "L3: grep" hoy es manual, no se hace automáticamente).
PRINCIPLES.md PARTE I § 3 (SRSI)	Source verification step de BGPD	SRSI exige "ejecutar grep/rg sobre el patrón del cambio. Confirmar matches antes y después." Es conceptualmente equivalente al paso 4 de BGPD.	Falta: integración con locators del handbook (hoy SRSI es ad-hoc, no guiado por L3 entries).
PRINCIPLES.md PARTE I § 4 (DDLp)	Edit Planning desde gaps	DDLp exige leer PLAN-VIVO § 3 (gaps) y § 4 (solicitudes) antes de planear. Es conceptualmente similar a "convertir evidence en edit plan".	Falta: edit plan estructurado con old/new pairs y Γ declarations.
LOOPS.md	Loop format + fixed_check	Define 12 campos canónicos del loop incluyendo fixed_check (comando reproducible). El paper exige edit blocks con verificación; fixed_check cumple ese rol.	Falta: Γ = (will_modify, will_add, will_remove) para alimentar Resync.
worker-ejecutor SKILL.md	Executor separado del planner	Define scope mínimo, commit atómico, SRSI antes de fix, fixed_check primero, stop conditions. Es el rol del executor del paper.	Falta: separation estricta planner vs executor, syntax gate configurable.
reviewer-calidad SKILL.md	Critic en el Propose-Review-Mapping loop	Cumple el rol del reviewer en Phase II. Audita fixed_check como prompt.	Falta: integración con handbook generation (hoy solo audita loops, no clasifica funciones).

Componente harness	Concepto paper	Mapeo	Falta para cumplir
PRINCIPLES.md PARTE I § 2 (R-DOS-PLANOS)	Progressive disclosure	Exige que el plano mayor (PLAN-VIVO) no se sobrescriba y se archive al crecer. Es conceptualmente progressive disclosure: plano mayor = L1, plano menor = L2/L3.	Falta: enforcement automático (propuesto en patch 1.5 de la auditoría previa).
AGENT-PROTOCOL.md § 4.7 Triggers	Skill routing por task_type	Tabla "Tipo de cambio → Skill obligatorio" es conceptualmente el stage selection de BGPD: cada task_type mapea a skill/MCPs.	Falta: routing automático basado en L1/L2 del handbook.

Tabla 5. Mappings exactos entre componentes del harness y conceptos del paper.

4.2 Conclusión de la sección

El harness jcode es un **andamiaje correcto pero incompleto**. Tiene las plantillas (BEHAVIOR-INDEX, STATE-REGISTERS), la política (skill orient/ BGPD), la filosofía (PRINCIPLES progressive disclosure, SRSI source verification, DDLP edit planning) y los roles (worker-ejecutor = executor, reviewer-calidad = critic). Lo que le falta es la **automatización** que conecte estos componentes en un pipeline end-to-end como el que el paper especifica. Las secciones siguientes proponen los patches concretos para cerrar cada gap.

5. Gap 1 — Construction Pipeline ausente

Severidad: Critical. El construction pipeline es el corazón del paper. Sin él, BEHAVIOR-INDEX.md y STATE-REGISTERS.md son plantillas que el usuario debe llenar manualmente, lo que viola el principio fundamental del paper: "organizes implementation knowledge around system behaviors and links each behavior to the corresponding source code" (§ 1). Un handbook manual se desincroniza al primer commit y pierde toda utilidad.

5.1 Lo que el paper exige

El paper requiere un pipeline de 3 fases (§ 3.2, Figura 2):

```
Phase I. Static Fact Extraction (determinístico, cero LLM)
- AST adapters parsean el repo
- Extraen: functions, boundaries externos, source locations,
  signatures, call edges
- Program graph G: solo calls que resuelven a función interna
  o boundary nombrado
- Unresolved calls: se loguean, NO se asignan a targets adivinados
- Output: program_graph.json

Phase II. Behavioral Organization (LLM-assisted, Propose-Review-Mapping)
- function-as-leaf: usa S0 seed + call-graph context
  → proponer function-to-stage assignments
  → iterative review hasta estabilizar
- file-as-leaf: file cards + G → infer S, organizar files por stage
- Output: behavioral_mapping.json (S, U_g)

Phase III. Hierarchical Synthesis (LLM para descripciones; paths/locators son facts)
- Convierte (S, U_g) en document tree D (L1, L2, L3 entries)
- Genera vista Z (state-registers view)
- Valida cada L3 locator contra R
- Empaqueta estado de resync K_g + cache B
- Output: .jcode/handbook/references/{overview,index,registers,stages/*}.md
```

5.2 Lo que el harness tiene hoy

El harness tiene las **plantillas destino** pero ningún script que las genere o mantenga:

```
.jcode/BEHAVIOR-INDEX.md      # plantilla vacía con placeholders B-XXX
.jcode/STATE-REGISTERS.md     # plantilla vacía con placeholders <tabla>.<col>
.jcode/skills/orient/SKILL.md  # declara política BGPD pero sin automation
.jcode/skills/arquitecto-proyecto/SKILL.md # rol pero sin pipeline

# NO EXISTE:
# .jcode/lib/handbook_builder.py   (Phase I/II/III)
# .jcode/lib/handbook_resync.py    (Resync)
# .jcode/handbook/references/      (output del pipeline)
# .jcode/handbook/program_graph.json (G)
# .jcode/handbook/behavioral_mapping.json (S, U_g)
# .jcode/handbook/cache/          (B)
```

5.3 Impacto de no tener pipeline

Sin pipeline, los artefactos BEHAVIOR-INDEX y STATE-REGISTERS se llenan manualmente al iniciar el proyecto y **nunca se actualizan**. Esto produce tres fallos cascada: (1) cuando el agente consulta el handbook durante BGPD, obtiene información obsoleta al primer refactor; (2) la skill orient/ pasa a ser engañosa porque dirige al agente a entradas que ya no existen; (3) el compliance score del harness no mide si el handbook está sincronizado, solo si el agente "declaró" haberlo leído. El resultado es que el harness parece cumplir el paper en filosofía pero no entrega el valor que el paper demostró (+18.9% win rate en localization, -12.7% planner tokens).

6. Gap 2 — Resync automático ausente

Severidad: Critical. El Resync es lo que mantiene el handbook sincronizado con el repo a lo largo del tiempo. Sin él, incluso un handbook perfectamente generado en T_0 se vuelve incorrecto en T_1 tras el primer commit que toque código. El paper es explícito: "any non-empty diff triggers handbook resynchronization" (§ 3.3, Algorithm 1 línea 6-7).

6.1 Lo que el paper exige (Appendix B.4)

El procedimiento `Resync_g(H, R, R', Δ, Γ)` tiene 3 sub-pasos:

- (1) Version alignment
 - Reparse R' , construir G'
 - Comparar G' con G usando Δ
 - Identificar added, removed, modified units
 - function-mode: match por body fingerprints (ignora line numbers)
rename = match body bajo signature line
 - file-mode: file-set differences + content hashes
 - path rename = 1 removal + 1 addition
- (2) Scoped update (si S sigue siendo válido)
 - $S' = S$
 - Actualizar $U_g \rightarrow U_g'$ (solo affected parts)
 - function-mode: remove deleted memberships, carry forward matched, reclassify new/changed, refresh region anchors, source hashes, within-stage order, unmapped status
 - file-mode: remove obsolete cards/assignments, refresh cards for new/changed files, assign new files, reorganize stages, update coverage record $Y \rightarrow Y'$
 - Regenerar L3 entries afectadas + enclosing L2/L1 ancestors + dependent state registers
 - Reusar unaffected generation outputs de cache B
 - Si S ya no acomoda: full rebuild (rerun Algorithm 2)
- (3) Conservative handling
 - Source no parseable: NO se adivina
 - L3 entry con locator/hash no revalidable: marcada FROZEN, excluida de localization hasta refresh
 - function-mode: new/changed no clasificables $\rightarrow$ unmapped in U_{function}'
 - file-mode: files sin card/assignment válida $\rightarrow$ coverage record Y'

Model calls limitadas a 4 categorías semánticas:

- classification (stage/function decisions)
- file assignment (file-to-stage decisions)
- within-stage organization (file grouping)
- description revision (regenerating cards/nodes/registers)

6.2 Lo que el harness tiene hoy

El hook `turnend.sh` hace **auditoría post-commit** pero nunca toca el handbook. Específicamente, líneas 40-60:

```
# turnend.sh — auditoría post-commit actual
source "$JCODE_DIR/lib/git_age.sh"
last_commit_age=$(last_commit_minutes)
```

```
if [[ $last_commit_age -le 5 ]]; then
  # commit reciente — verificar que la última entrada de §6 menciona
  # algún loop related al commit
  last_commit_msg=$(last_commit_subject)
  section6_hit=$(grep -c "L-" "$JCODE_DIR/iterations/PLAN-VIVO.md" 2>/dev/null)
  if [[ $section6_hit -le 4 ]]; then
    echo "[turnend] ▲ Commit reciente sin update de PLAN-VIVO §6"
  fi
fi
# NUNCA llama a handbook_resync
# NUNCA toca BEHAVIOR-INDEX.md
# NUNCA toca STATE-REGISTERS.md
```

El gap es doble: (a) no hay script `handbook_resync.py` que implemente el procedimiento del paper; (b) el hook `turnend.sh` no lo invoca aunque existiera. Ambos se resuelven con los patches 4 y 5 (secciones 18 y 19).

7. Gap 3 — Behavior-Implementation Alignment

Severidad: Critical. El paper exige que cada L3 locator activo resuelva al repo actual. Si no puede revalidarse, la entrada se marca **frozen** y se excluye de localization (§ 3.1 regla 2, Appendix B.4 Conservative handling). Sin esto, el handbook miente al agente: lo dirige a archivos que ya no existen, funciones renombradas, o líneas que se movieron.

7.1 Lo que el paper exige

Cada L3 entry debe contener un **source locator** con: file path (exacto), anchor opcional (function name o region), y source excerpt actual. El locator se valida contra R en dos momentos: (1) al construir el handbook (Phase III grounding and failure handling); (2) al resync (Resync Conservative handling). Si la validación falla, la entry se **freeza** y se excluye de BGPD hasta que se refresque.

```
# L3 entry canónica (paper §3.1 + Appendix D.1.4)
{
  "schema_version": 1,
  "type": "single", # o "multi" para regions
  "locator_role": "<one-line role tag, 6-15 words>",
  "stage_context": "<2-4 sentences>",
  "synopsis": "<2-4 sentences>",
  "interface": {
    "signature": "<full signature>",
    "params": [...],
    "reads_state": [...],
    "returns": "...",
    "side_effects": [...]
  },
  "execution_flow": [...],
  "design_decisions": [...],
  "relations": {
    "callers": [...],
    "core_callees": [...],
    "register_interactions": [...]
  },
  "locator": {
    "file": "src/auth/handlers.py",
    "anchor": "LoginHandler.post",
    "line_range": [142, 178],
    "source_hash": "sha256:abc123..."
  }
}
```

7.2 Lo que el harness tiene hoy

BEHAVIOR-INDEX.md lista "Archivos" como strings simples sin locators verificables:

```
# BEHAVIOR-INDEX.md — plantilla actual
## B-001 — Registro de nuevo usuario
- **Comportamiento**: Usuario nuevo completa formulario...
- **Archivos**:
  - `backend/internal/handlers/auth.go`
  - `frontend/src/pages/register.astro`
```

```
- `db/migrations/003_create_users.sql`  
# ↑ paths como strings, sin line_range, sin anchor,  
# sin source_hash, sin verificación de existencia
```

Si un refactor renombra `auth.go` a `auth_handler.go`, el BEHAVIOR-INDEX queda apuntando a un path inexistente. El agente que siga BGPD caerá en el anti-patrón que el paper Section 1 describe: "coding agents face an additional constraint from limited input context, which prevents them from examining all relevant code at once. They must therefore explore the repository iteratively and may still overlook scattered or rarely executed paths". El handbook debería **prevenir** eso, no causarlo.

7.3 Mecanismo de freeze que falta

El paper exige que las entries no revalidables se marquen frozen y se excluyan de localization. El harness no tiene este mecanismo. El patch 4 (sección 18) propone un archivo `handbook/frozen_entries.json` que el Resync actualiza y que BGPD consulta antes de retornar candidates.

8. Gap 4 — BGPD sin source verification

Severidad: High. La skill orient/ del harness declara la política BGPD de 4 niveles (L1 → L2 → Z → L3) pero el paso 4 (source verification) **nunca se ejecuta automáticamente**. El agente puede leer BEHAVIOR-INDEX, identificar paths candidatos, y construir un edit plan sin haber verificado que esos paths existen. Esto es exactamente el problema que el paper resuelve.

8.1 Lo que el paper exige (§ 3.3.1, Appendix B.1 paso 4)

Tras operar sobre el handbook H, el agente debe abrir el repo R, resolver cada locator candidato, y retener solo los sites que siguen siendo relevantes al request q. El output es el **evidence set verificado E_bq** donde cada record contiene: file path, optional function/region anchor, y current source excerpt.

```
# BGPD paso 4 — Source verification (paper Appendix B.1)
def verify_candidates(candidates, repo_root, request_q):
    """Abre R, resuelve cada locator, descarta los que no matchean."""
    verified = []
    for cand in candidates:
        path = os.path.join(repo_root, cand.file)
        if not os.path.exists(path):
            # L3 entry está FROZEN — excluir de localization
            mark_frozen(cand.locator_id)
            continue
        source = read_file(path)
        if cand.anchor:
            excerpt = extract_around_anchor(source, cand.anchor,
                                           cand.line_range)

            if not excerpt:
                # anchor no encontrado — renombrado o borrado
                mark_frozen(cand.locator_id)
                continue
        else:
            excerpt = extract_lines(source, cand.line_range)
        # Verificar source_hash si existe
        if cand.source_hash:
            current_hash = sha256(excerpt.encode())
            if current_hash != cand.source_hash:
                # Contenido cambió — marcar para resync
                mark_stale(cand.locator_id)
        # Verificar relevancia contra q (heurística LLM opcional)
        if is_relevant(excerpt, request_q):
            verified.append({
                "file": cand.file,
                "anchor": cand.anchor,
                "line_range": cand.line_range,
                "excerpt": excerpt
            })
    return verified
```

8.2 Lo que el harness tiene hoy

La skill orient/ Fase 2-B paso 1 dice: "L3: Hacer grep/rg en los archivos para confirmar existencia". Esto es una **instrucción** al agente, no un **mecanismo** que se ejecute automáticamente. El agente puede omitirlo (y lo hace, según la experiencia reportada por el usuario). El patch 5 (sección 19) propone agregar un script handbook_verify.py que el agente DEBE invocar antes de declarar fixed_check, con hook posttool.sh que detecte la omisión.

9. Gap 5 — Program Graph ausente

Severidad: High. El program graph G es el input de Phase II y el motor de BGPD paso 3 (call-relation expansion). Sin G, no se puede hacer expansion automática de candidates沿着 call relations, ni identificar coupled stages via shared state. El harness usa grep/rg ad-hoc que es exactamente el baseline del paper (la condición que el paper mejora).

9.1 Lo que el paper exige (Phase I, § 3.2)

Phase I produce un **program graph G** con 4 tipos de facts:

```
# Program graph G (paper §3.2, Appendix A)
{
  "functions": [
    {
      "qualname": "Terminus2._run_agent_loop",
      "file": "terminus_2.py",
      "line_range": [1427, 1560],
      "signature": "(self) -> None",
      "body_hash": "sha256:..."
    },
    ...
  ],
  "boundaries": [
    # external named boundaries (API surfaces, library entry points)
    { "name": "OpenAI.chat.completions.create",
      "type": "external_api" }
  ],
  "call_edges": [
    { "caller": "Terminus2._run_agent_loop",
      "callee": "Terminus2._should_exit",
      "line": 1432,
      "resolved": true },
    ...
  ],
  "state_accesses": [
    { "function": "Terminus2._run_agent_loop",
      "attribute": "self._pending_completion",
      "access": "write",
      "line": 1440 },
    ...
  ],
  "unresolved_calls_log": [
    # calls que no resuelven a función interna ni boundary nombrado
    # se loguean, NO se asignan a targets adivinados
  ]
}
```

9.2 Métricas del paper (Table 2)

El paper reporta el tamaño de G para los dos harnesses evaluados:

Harness	Lang	Leaf mode	Source files	Functions	Boundaries	Call edges	Handbook output
Terminus-2	Python	function-as-leaf	6	103	41	257	20 stages, 106 L3 entries, 10 state registers
Codex	Rust	file-as-leaf	2,267	34,363	4,016	159,960	140 stages, 2,267 L3 entries, 62 state registers

Tabla 6. Phase I static facts para los dos harnesses del paper (Table 2).

El harness jcode está más cerca del caso Terminus-2 (Python, compacto, function-as-leaf viable) que del caso Codex (Rust monorepo, file-as-leaf necesario). El patch 1 (sección 16) propone un Phase I mínimo usando el módulo ast de stdlib Python, extensible a otros lenguajes vía tree-sitter.

10. Gap 6 — Edit Planning estructurado

Severidad: Medium. El paper exige que el edit plan P contenga bloques con **old/new pairs** **byte-exact** y que cada edit tenga una **action declaration** en $\Gamma = (\Gamma_{\text{modify}}, \Gamma_{\text{add}}, \Gamma_{\text{remove}})$ a function granularity. Sin Γ , no se puede auditar si execution siguió el plan ni alimentar Resync con información estructurada.

10.1 Lo que el paper exige (§ 3.3.2, Appendix B.2)

```
# Edit plan P — formato del paper (Appendix B.2)
### EDIT 1
- file: src/terminus_2.py
- where: completion gate — third-mark confirmation
```old
if is_task_complete:
 if was_pending_completion:
 return
 else:
 prompt = observation
 continue
...
```new
if is_task_complete:
    if was_completion_confirmations >= 2:
        return
    else:
        prompt = observation
        continue
...

# Action declarations  $\Gamma$  (machine-readable, consumes Resync)
```json
{
 "will_modify": ["Terminus2._run_agent_loop",
 "Terminus2.__init__",
 "Terminus2._reset_per_run_state"],
 "will_add": [],
 "will_remove": []
}
```
```

10.2 Lo que el harness tiene hoy

LOOPS.md define 12 campos del loop incluyendo fixed_check (comando reproducible) y handoff_artifact. Pero NO define edit blocks con old/new pairs ni action declarations Γ . El template OP.md (procedimientos operacionales) tampoco los tiene. El patch 7 (sección 22) propone extender OP.md con los campos que el paper exige.

11. Gap 7 — Leaf mode ausente

Severidad: Medium. El paper define $g \in \{\text{function}, \text{file}\}$ como una decisión fija al construir el handbook. Determina la granularidad de las L3 entries y afecta todo el pipeline. El harness no tiene este concepto: BEHAVIOR-INDEX usa "archivos" informalmente, sin decidir si la leaf es función o archivo. Esto produce ambigüedad operacional.

11.1 Criterio del paper para elegir g (§ 3.2)

El paper da un criterio claro:

```
# Selección de leaf mode (paper §3.2)
if reliable_seed_skeleton_available and function_level_fits_budget:
    g = "function" # function-as-leaf
    # S0 seed + Phase II Propose-Review-Mapping
else:
    g = "file" # file-as-leaf
    # Sin S0, inferir S desde file cards + G

# Ejemplos del paper:
# Terminus-2: function-as-leaf (6 archivos, 103 funciones, S0 confiable)
# Codex: file-as-leaf (2,267 archivos, 34,363 funciones, function-level
#           excedería budget)
```

11.2 Recomendación para el harness jcode

Como el harness está diseñado para proyectos nuevos que arrancan desde cero, la recomendación es **defaultear a function-as-leaf** (los proyectos nuevos son pequeños al inicio, con S0 = task_class de PRINCIPILES PARTE II como seed) y **auto-promocionar a file-as-leaf** cuando el repo supere un umbral (ej: > 200 funciones o > 50 archivos). El patch 1 (sección 16) implementa este auto-detect.

12. Gap 8 — SKILL.md manifest canonical

Severidad: Low. El paper Figure 6 define un SKILL.md canonical que expone el handbook al planner como skill navegable. Las skills del harness (orient/, arquitecto-proyecto/) existen pero no siguen este layout. Esto es cosmético pero importante para interoperabilidad.

12.1 SKILL.md del paper (Figure 6)

```
# SKILL.md canonical del paper (Figure 6)
---
name: <harness>-handbook
description: >-
  Structural map of the harness, derived from its code.
  Use it when planning a change to find EVERY code site
  the change must touch -- it maps the code stage-by-stage
  and lists every state register together with all of its
  read/write locations.
---

# Harness Handbook: navigation guide

## Reference files
- references/overview.md
  -- whole-system orientation: lifecycle, main loop,
  how stages fit together. Read first.
- references/index.md
  -- the map: every stage (id, title, what it does,
  the leaves it covers) and every state register.
- references/registers.md
  -- for each state register: its purpose and EVERY place
  it is written and read, across the whole harness.
- references/stages/<id>.md
  -- one stage's prose plus a card per leaf (function or
  file): its <summary> line gives the exact code location
  (path:start-end), and the card holds the interface,
  behavior, relations, and source.

## How to use it (during planning, before you write your plan)
1. Read references/overview.md first.
2. Read references/index.md to identify stages, leaves,
  AND state registers your change involves.
3. For EVERY state register your change touches, read
  references/registers.md and note EVERY write and read site.
4. Open references/stages/<id>.md for detail.
5. For each site the handbook names, open the real code
  with read_file / search_file_content and locate precise lines.
6. Your plan must account for every site the handbook surfaced.
```

El patch 6 (sección 21) propone crear .jcode/skills/handbook/SKILL.md siguiendo este formato canónico, que reemplaza al BEHAVIOR-INDEX.md estático por el handbook generado automáticamente.

13. Blueprint de implementación por fases

Las 6 fases siguientes cierran los 8 gaps identificados. Cada fase tiene: objetivo, gaps que cierra, entregables, esfuerzo estimado, y dependencias. El orden respeta las dependencias técnicas (no se puede hacer Resync sin pipeline que genere el handbook que se va a resync).

| Fase | Nombre | Gaps cerrados | Entregables | Esfuerzo | Depends |
|------|---|-------------------------|--|----------|---------|
| F1 | Phase I — Static Fact Extraction | Gap 5 | <code>.jcode/lib/handbook_builder.py</code> con <code>extract_static_facts()</code> . AST parser Python (stdlib), extensible. Output: <code>program_graph.json</code> . Auto-detect leaf mode. | 3-5 días | Ninguna |
| F2 | Phase II — Behavioral Organization | Gap 1 (parcial) | Propose-Review-Mapping loop con LLM (z-ai-web-dev-sdk). Prompts del Appendix D.1.1. Output: <code>behavioral_mapping.json</code> con (S, U _g). | 5-7 días | F1 |
| F3 | Phase III — Hierarchical Synthesis | Gap 1 (completo), Gap 7 | Genera <code>L1 overview.md</code> , <code>L2 stages/<id>.md</code> , <code>L3 entries</code> con locators, registers.md. Output: <code>.jcode/handbook/references/*.md</code> + cache B. | 3 días | F2 |
| F4 | Resync hook post-commit | Gap 2, Gap 3 (parcial) | <code>handbook_resync.py</code> + modificar <code>turnend.sh</code> . Implementa <code>version align</code> + <code>scoped update</code> + <code>conservative handling</code> + <code>freeze</code> . Output: <code>frozen_entries.json</code> . | 4 días | F3 |
| F5 | BGPD con source verification | Gap 4, Gap 3 (completo) | <code>handbook_verify.py</code> + modificar <code>skill orient/SKILL.md</code> . Verifica locators contra filesystem, descarta frozen. Hook posttool detecta omisión. | 2 días | F4 |
| F6 | Edit Planning con Γ declarations | Gap 6 | Modificar <code>OP.md template</code> con campos <code>will_modify/will_add/will_remove</code> . Worker-ejecutor skill actualizada para emitir Γ . | 3 días | F5 |

Tabla 7. Blueprint de implementación en 6 fases.

13.1 Timeline resumido

| | | |
|--|------------------------------|---------------------------------------|
| Semana 1: | F1 (Phase I) | → 3-5 días |
| Semana 2: | F2 (Phase II) | → 5-7 días |
| Semana 3: | F3 (Phase III) | → 3 días |
| | F4 (Resync) | → 4 días (paralelizable con F3 final) |
| Semana 4: | F5 (BGPD verify) | → 2 días |
| | F6 (Edit planning Γ) | → 3 días |
| | Testing + docs | → 2 días |
| <hr/> | | |
| Total: ~3-4 semanas (1 dev full-time) | | |
| Después de F3: handbook generado automáticamente | | |
| Después de F4: handbook se mantiene sincronizado | | |
| Después de F5: BGPD verifica contra repo real | | |

Después de F6: edit plans estructurados alimentan Resync

13.2 Priorización P0/P1/P2

| Prioridad | Criterio | Fases | Justificación |
|-----------|--|---------------------------|--|
| P0 | Crítico — sin esto el harness miente al usuario | F1 + F2 + F3 + F4 | Cierra 5 de 8 gaps (1, 2, 3 parcial, 5, 7). Sin esto, BEHAVIOR-INDEX y STATE-REGISTERS son decoración. |
| P1 | Alto — sin esto el handbook pierde valor operacional | F5 + F6 | Cierra 3 de 8 gaps (4, 6, 3 completo). Habilita el workflow BGPD completo del paper. |
| P2 | Medio — refinamientos y métricas | Tests, CI, observabilidad | Validación de los patches, integración con visualizer, métricas de+18.9% win rate. |

Tabla 8. Priorización P0/P1/P2 del blueprint.

14. Patch 1 — handbook_builder.py (Phase I)

Esqueleto del construction pipeline. Implementa Phase I (Static Fact Extraction) usando el módulo ast de stdlib Python para proyectos Python, con estructura extensible a otros lenguajes vía adapters. Output: program_graph.json con el formato del paper (functions, boundaries, call_edges, state_accesses, unresolved_calls_log).

14.1 Código — jcode/lib/handbook_builder.py

```
#!/usr/bin/env python3
"""
handbook_builder.py — Construction pipeline del Harness Handbook.
Implementa Phase I/II/III del paper arXiv:2607.13285v1.

Usage:
python3 jcode/lib/handbook_builder.py phase1 --repo .
python3 jcode/lib/handbook_builder.py phase2 --repo .
python3 jcode/lib/handbook_builder.py phase3 --repo .
python3 jcode/lib/handbook_builder.py build --repo . # all 3
"""

import ast, os, sys, json, hashlib, argparse
from pathlib import Path

# =====
# Phase I — Static Fact Extraction (determinístico, cero LLM)
# =====

class PythonAdapter:
    """AST adapter para Python. Extender con RustAdapter, GoAdapter, etc."""

    LANG = "python"
    FILE_EXT = ".py"

    def parse_file(self, path: Path) -> dict:
        """Extrae functions, call edges, state accesses de un archivo."""
        try:
            src = path.read_text(encoding="utf-8")
            tree = ast.parse(src, filename=str(path))
        except (SyntaxError, UnicodeDecodeError) as e:
            return {"file": str(path), "error": str(e),
                    "functions": [], "calls": [], "state": []}

        functions = []
        calls = []
        state = []

        for node in ast.walk(tree):
            if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
                func = self._extract_function(node, path, src)
                functions.append(func)
                calls.extend(self._extract_calls(node, func["qualname"]))
                state.extend(self._extract_state(node, func["qualname"]))

        return {
            "file": str(path),
            "functions": functions,
```

```

        "calls": calls,
        "state": state
    }

def _extract_function(self, node, path, src) -> dict:
    qualname = self._qualname(node)
    body_src = ast.get_source_segment(src, node) or ""
    body_hash = hashlib.sha256(body_src.encode()).hexdigest()
    sig = ast.get_source_segment(src, node.args)
    return {
        "qualname": qualname,
        "file": str(path),
        "line_range": [node.lineno, node.end_lineno or node.lineno],
        "signature": f"({sig})" if sig else "()",
        "body_hash": f"sha256:{body_hash}",
    }

def _qualname(self, node) -> str:
    """Build qualified name: Class.method or function."""
    # Walk parents to build qualname (simplified)
    return node.name # TODO: walk parent ClassDef

def _extract_calls(self, node, caller_qualname) -> list:
    calls = []
    for child in ast.walk(node):
        if isinstance(child, ast.Call):
            callee = self._call_target(child.func)
            if callee:
                calls.append({
                    "caller": caller_qualname,
                    "callee": callee,
                    "line": child.lineno,
                    "resolved": False # resolved in _resolve_calls
                })
    return calls

def _call_target(self, func_node) -> str | None:
    if isinstance(func_node, ast.Name):
        return func_node.id
    if isinstance(func_node, ast.Attribute):
        return f"{self._attr_chain(func_node.value)}.{func_node.attr}"
    return None

def _attr_chain(self, node) -> str:
    if isinstance(node, ast.Name):
        return node.id
    if isinstance(node, ast.Attribute):
        return f"{self._attr_chain(node.value)}.{node.attr}"
    return "_"

def _extract_state(self, node, func_qualname) -> list:
    """Extract self.X read/write accesses."""
    accesses = []
    for child in ast.walk(node):
        if isinstance(child, ast.Attribute) and \
            isinstance(child.value, ast.Name) and \
            child.value.id == "self":
            # Determine read vs write by parent

```

```

        access = "read" # default
        # Simplified: if appears in Assign target, it's write
        accesses.append({
            "function": func_qualname,
            "attribute": f"self.{child.attr}",
            "access": access,
            "line": child.lineno
        })
    return accesses

def extract_static_facts(repo_root: str, language: str = "python") -> dict:
    """Phase I entrypoint. Returns program graph G."""
    adapter = _get_adapter(language)
    repo = Path(repo_root)
    source_dirs = _read_source_dirs(repo)

    all_functions = []
    all_calls = []
    all_state = []
    unresolved = []
    files_scanned = 0

    for sd in source_dirs:
        for path in repo.glob(f"{sd}/**/*{adapter.FILE_EXT}"):
            if _is_excluded(path):
                continue
            result = adapter.parse_file(path)
            files_scanned += 1
            if "error" in result:
                unresolved.append({"file": str(path),
                                   "error": result["error"]})
                continue
            all_functions.extend(result["functions"])
            all_calls.extend(result["calls"])
            all_state.extend(result["state"])

    # Resolve calls: match callee name to internal function qualname
    qualnames = {f["qualname"] for f in all_functions}
    resolved_edges = []
    for call in all_calls:
        callee = call["callee"]
        # Try exact match, then suffix match (Class.method)
        if callee in qualnames:
            call["resolved"] = True
            resolved_edges.append(call)
        elif any(q.endswith("." + callee.split(".")[-1]) for q in qualnames):
            call["resolved"] = True
            resolved_edges.append(call)
        else:
            call["resolved"] = False
            unresolved.append({"type": "unresolved_call",
                               "callee": callee,
                               "caller": call["caller"],
                               "line": call["line"]})

    # Auto-detect leaf mode
    leaf_mode = "function" if len(all_functions) <= 200 else "file"

```



```

return {
    "schema_version": 1,
    "language": language,
    "leaf_mode": leaf_mode,
    "files_scanned": files_scanned,
    "functions": all_functions,
    "boundaries": [], # TODO: detect external API boundaries
    "call_edges": resolved_edges,
    "state_accesses": all_state,
    "unresolved_calls_log": unresolved,
}

def _get_adapter(language: str):
    adapters = {"python": PythonAdapter}
    if language not in adapters:
        raise ValueError(f"No adapter for {language}. Available: {list(adapters)}")
    return adapters[language]()

def _read_source_dirs(repo: Path) -> list:
    """Read [workspace] source_dirs from config.toml."""
    # Simplified — implement proper TOML parse in production
    return ["src", "tests"]

def _is_excluded(path: Path) -> bool:
    parts = str(path)
    return any(x in parts for x in [
        "node_modules", ".git", "dist", "build",
        "__pycache__", ".venv", ".jcode/iterations/archive"
    ])

# =====
# CLI
# =====
if __name__ == "__main__":
    p = argparse.ArgumentParser(description="Handbook builder")
    p.add_argument("command", choices=["phase1", "phase2", "phase3", "build"])
    p.add_argument("--repo", default=".", help="Repo root")
    p.add_argument("--language", default="python")
    args = p.parse_args()

    if args.command == "phase1":
        G = extract_static_facts(args.repo, args.language)
        out = Path(args.repo) / ".jcode" / "handbook" / "program_graph.json"
        out.parent.mkdir(parents=True, exist_ok=True)
        out.write_text(json.dumps(G, indent=2))
        print(f"[ok] Phase I: {len(G['functions'])} functions, "
              f"{len(G['call_edges'])} resolved calls, "
              f"leaf_mode={G['leaf_mode']}")
        print(f"[ok] Output: {out}")
    elif args.command in ("phase2", "phase3", "build"):
        print(f"[todo] {args.command} — ver patches 2 y 3")

```

Notas de implementación: (1) el adapter Python usa ast de stdlib, cero dependencias externas; (2) la función `_qualname` está simplificada — en producción debe walk parent ClassDef para construir Class.method; (3) la detección de read vs write en `_extract_state` está simplificada — en producción debe inspeccionar el parent Assign node; (4) los adapters para Rust, Go, Java, TypeScript requieren tree-sitter (ver sección 14.2).

14.2 Extensión a otros lenguajes

Para soportar multi-lenguaje, implementar adapters con tree-sitter:

```
# Estructura de adapters (futuro)
.jcode/lib/adapters/
├── __init__.py
├── python_adapter.py    ← usa stdlib ast
├── rust_adapter.py      ← usa tree-sitter-rust
├── go_adapter.py        ← usa tree-sitter-go
├── typescript_adapter.py ← usa tree-sitter-typescript
└── java_adapter.py      ← usa tree-sitter-java

# Cada adapter implementa la interfaz:
class LanguageAdapter:
    LANG: str
    FILE_EXT: str
    def parse_file(self, path: Path) -> dict: ...
    # Output dict tiene mismo schema que PythonAdapter.parse_file()
```

15. Patch 2 — Phase II con LLM

Implementa Phase II (Behavioral Organization) usando el loop Propose-Review-Mapping del paper. Toma el program graph G de Phase I y produce el behavioral mapping (S, U_g) donde S es el stage skeleton y U_g es la organización de unidades por stage. Usa los prompts del Appendix D.1.1 del paper.

15.1 Código — `jcode/lib/handbook_phase2.py`

```
#!/usr/bin/env python3
"""
handbook_phase2.py — Phase II: Behavioral Organization.
Propose-Review-Mapping loop con LLM.
"""

import json, sys, os
from pathlib import Path

# z-ai-web-dev-sdk (skill LLM) para llamadas al modelo
sys.path.insert(0, os.path.expanduser("~/local/share/z-ai-web-dev-sdk/python"))
try:
    from zai import ZaiClient
    LLM = ZaiClient()
except ImportError:
    LLM = None
    print("[warn] z-ai-web-dev-sdk no disponible, usando modo stub",
          file=sys.stderr)

# Seed skeleton para jcode harness (S0)
# Basado en PRINCIPLES.md PARTE II task_class + execution stages típicos
JCODE_SEED_SKELETON = {
    "stages": [
        {"id": "init", "title": "Session Init",
         "purpose": "State init + integrity check"},
        {"id": "interpret", "title": "Interpretation",
         "purpose": "Task triage + skill/MCP selection"},
        {"id": "plan", "title": "Loop Declaration",
         "purpose": "Declare loop with fixed_check + budget"},
        {"id": "execute", "title": "Execution",
         "purpose": "Apply changes within scope"},
        {"id": "verify", "title": "Verification",
         "purpose": "Run fixed_check + evidence"},
        {"id": "handoff", "title": "Handoff",
         "purpose": "Update PLAN-VIVO + closeout"},
    ]
}

CLASSIFY_PROMPT = """You are analyzing one function from a codebase and
classifying it into one or more stages of a hand-authored data-flow
skeleton.

GRANULARITY
- "function": the entire function is a single narrative unit.
  Use for short, cohesive functions (<=30 lines or single purpose).
- "region": the function contains multiple distinct narrative steps
  and should be split into 2-10 regions.
```

STAGE ASSIGNMENT

- Pick stage IDs ONLY from the list provided.
- CROSS-CUTTING utility functions (small utilities called from many places) go to "crosscut" stage.

PURPOSE FIELD (60-150 words, structured across 5 aspects)

1. ACTION: what the function does, concretely
2. INPUTS / STATE READ: arguments + attributes read
3. OUTPUTS / STATE WRITTEN: return value + attributes mutated
4. WHEN INVOKED: who calls it, under what condition
5. NON-OBVIOUS: retry logic, fallback paths, design choices

OUTPUT: ONLY a JSON object inside a ```json fenced block:

```
{
  "qualname": "<exact qualname>",
  "purpose": "<60-150 word 5-aspect description>",
  "granularity": "function" | "region",
  "function_assignments": ["stage-id", ...],
  "regions": null,
  "file": "<file>",
  "line_range": [<start>, <end>]
}
```

AVAILABLE STAGES:

```
{stages}
```

FUNCTION METADATA:

```
{metadata}
```

CALLER/CALLEE CONTEXT:

```
{context}
```

SOURCE:

```
```python
{source}
```
"""
```

```
def classify_function(func: dict, context: dict, stages: list) -> dict:
```

```
    """LLM call to classify one function into stages."""
```

```
    if LLM is None:
```

```
        # Stub: assign to "execute" by default
```

```
        return {
```

```
            "qualname": func["qualname"],
```

```
            "purpose": "(stub) " + func["qualname"],
```

```
            "granularity": "function",
```

```
            "function_assignments": ["execute"],
```

```
            "regions": None,
```

```
            "file": func["file"],
```

```
            "line_range": func["line_range"]
```

```
        }
```

```
    prompt = CLASSIFY_PROMPT.format(
```

```
        stages=json.dumps(stages, indent=2),
```

```
        metadata=json.dumps({k: v for k, v in func.items()
```

```
            if k != "body_hash"}, indent=2),
```

```
        context=json.dumps(context, indent=2),
```

```
        source=_read_source(func)
```

```
)
```

```

resp = LLM.chat.completions.create(
    model="glm-4.6",
    messages=[{"role": "user", "content": prompt}],
    temperature=0.2
)
return _extract_json(resp.choices[0].message.content)

def _read_source(func: dict) -> str:
    """Read source lines from file."""
    path = Path(func["file"])
    if not path.exists():
        return f"(file not found: {path})"
    lines = path.read_text(encoding="utf-8").splitlines()
    start, end = func["line_range"]
    return "\n".join(lines[start-1:end])

def _extract_json(text: str) -> dict:
    """Extract JSON from ```json fenced block."""
    import re
    m = re.search(r"```json\s*(\{.*?\})\s*```", text, re.DOTALL)
    if not m:
        raise ValueError(f"No JSON block in response: {text[:200]}")
    return json.loads(m.group(1))

def behavioral_organization(program_graph: dict,
                           seed_skeleton: dict = None) -> dict:
    """Phase II entrypoint. Returns (S, U_g)."""
    if program_graph["leaf_mode"] == "function":
        return _organize_function_mode(program_graph, seed_skeleton)
    else:
        return _organize_file_mode(program_graph)

def _organize_function_mode(pg: dict, seed: dict) -> dict:
    """function-as-leaf: use S0 + Propose-Review-Mapping."""
    S = seed or JCODE_SEED_SKELETON
    stages = [{"id": s["id"], "title": s["title"],
                "purpose": s["purpose"]} for s in S["stages"]]
    assignments = []
    for func in pg["functions"]:
        context = _build_context(func, pg)
        result = classify_function(func, context, stages)
        assignments.append(result)
    return {
        "schema_version": 1,
        "leaf_mode": "function",
        "stage_skeleton": S,
        "function_assignments": assignments,
        "coverage_record": {
            "unmapped_functions": [a["qualname"] for a in assignments
                                   if not a["function_assignments"]]
        }
    }

def _organize_file_mode(pg: dict) -> dict:
    """file-as-leaf: infer S from file cards + G."""
    # TODO: implementar con prompts del Appendix D.1.3
    raise NotImplementedError("file-as-leaf Phase II — ver patch futuro")

```

```
def _build_context(func: dict, pg: dict) -> dict:
    """Build caller/callee context for a function."""
    callers = [c["caller"] for c in pg["call_edges"]
               if c["callee"] == func["qualname"]]
    callees = [c["callee"] for c in pg["call_edges"]
               if c["caller"] == func["qualname"]]
    return {"callers": callers[:5], "callees": callees[:5]}

if __name__ == "__main__":
    pg_path = Path(".jcode/handbook/program_graph.json")
    pg = json.loads(pg_path.read_text())
    result = behavioral_organization(pg)
    out = Path(".jcode/handbook/behavioral_mapping.json")
    out.write_text(json.dumps(result, indent=2))
    print(f"[ok] Phase II: {len(result['function_assignments'])} "
          f"functions assigned to {len(result['stage_skeleton']['stages'])} stages")
    print(f"[ok] Unmapped: {len(result['coverage_record']['unmapped_functions'])}")
    print(f"[ok] Output: {out}")
```

16. Patch 3 — Phase III synthesis

Convierte el behavioral mapping (S, U_g) de Phase II en el document tree D (L1-L2-L3) y la vista Z. Genera los archivos overview.md, index.md, registers.md, y stages/<id>.md con el formato canónico del paper (Figure 6).

16.1 Código — .jcode/lib/handbook_phase3.py

```
#!/usr/bin/env python3
"""
handbook_phase3.py — Phase III: Hierarchical Synthesis.
Convierte (S, Ug) en document tree D + vista Z.
"""

import json, os, sys
from pathlib import Path

sys.path.insert(0, os.path.expanduser("~/local/share/z-ai-web-dev-sdk/python"))
try:
    from zai import ZaiClient
    LLM = ZaiClient()
except ImportError:
    LLM = None

HANDBOOK_DIR = Path(".jcode/handbook/references")

def synthesize(behavioral_mapping: dict, program_graph: dict) -> dict:
    """Phase III entrypoint. Genera D + Z + cache B."""
    HANDBOOK_DIR.mkdir(parents=True, exist_ok=True)
    (HANDBOOK_DIR / "stages").mkdir(exist_ok=True)

    S = behavioral_mapping["stage_skeleton"]
    U = behavioral_mapping["function_assignments"]

    # 1. Generar L3 entries (una por function en function-mode)
    l3_entries = {}
    for func in U:
        entry = _generate_l3_entry(func, program_graph)
        l3_entries[func["qualname"]] = entry

    # 2. Generar L2 stage pages
    l2_pages = {}
    for stage in S["stages"]:
        stage_funcs = [f for f in U
                        if stage["id"] in f["function_assignments"]]
        l2_pages[stage["id"]] = _generate_l2_page(stage, stage_funcs, l3_entries)

    # 3. Generar L1 overview
    l1_overview = _generate_l1_overview(S, l2_pages)
    (HANDBOOK_DIR / "overview.md").write_text(l1_overview)

    # 4. Generar index.md
    index_md = _generate_index(S, U)
    (HANDBOOK_DIR / "index.md").write_text(index_md)

    # 5. Generar registers.md (vista Z)
    registers = _extract_state_registers(program_graph, U)
    registers_md = _generate_registers_md(registers)
```

```

(HANDBOOK_DIR / "registers.md").write_text(registers_md)

# 6. Validar locators contra R
frozen = _validate_locators(l3_entries)
if frozen:
    (HANDBOOK_DIR.parent / "frozen_entries.json").write_text(
        json.dumps(frozen, indent=2))
    print(f"[warn] {len(frozen)} entries frozen (locators no validan)",
          file=sys.stderr)

# 7. Empaquetar estado de resync K_g + cache B
k_g = {
    "leaf_mode": behavioral_mapping["leaf_mode"],
    "program_graph_hash": _hash_pg(program_graph),
    "stage_skeleton": S,
    "config": {"seed_skeleton": "jcode_default"},
}
cache_b = {"l3_entries": l3_entries, "l2_pages": l2_pages,
            "l1_overview": l1_overview}
(HANDBOOK_DIR.parent / "K_g.json").write_text(json.dumps(k_g, indent=2))
(HANDBOOK_DIR.parent / "cache_B.json").write_text(
    json.dumps(cache_b, indent=2, default=str))

return {"D": {"L1": l1_overview, "L2": l2_pages, "L3": l3_entries},
        "Z": registers, "frozen": frozen}

def _generate_l3_entry(func: dict, pg: dict) -> dict:
    """Generate L3 entry con source locator verificable."""
    locator = {
        "file": func["file"],
        "anchor": func["qualname"],
        "line_range": func["line_range"],
        "source_hash": _hash_source_region(func["file"], func["line_range"])
    }
    return {
        "schema_version": 1,
        "type": "single",
        "locator_role": func.get("purpose", "").split(".")[0][:100],
        "stage_context": func.get("purpose", ""),
        "synopsis": func.get("purpose", ""),
        "interface": {
            "signature": func.get("signature", "()"),
            "params": [], "reads_state": [],
            "returns": "see source", "side_effects": []
        },
        "execution_flow": [], "design_decisions": [],
        "relations": {
            "callers": [c["caller"] for c in pg["call_edges"]
                        if c["callee"] == func["qualname"]][:5],
            "core_callees": [c["callee"] for c in pg["call_edges"]
                             if c["caller"] == func["qualname"]][:5],
            "register_interactions": _register_interactions(
                func["qualname"], pg)
        },
        "locator": locator
    }

def _hash_source_region(file_path: str, line_range: list) -> str:

```



```

    """SHA256 of source region for locator validation."""
    import hashlib
    p = Path(file_path)
    if not p.exists():
        return "sha256:FILE_NOT_FOUND"
    lines = p.read_text(encoding="utf-8").splitlines()
    start, end = line_range
    region = "\n".join(lines[start-1:end])
    return f"sha256:{hashlib.sha256(region.encode()).hexdigest()}"

def _register_interactions(qualname: str, pg: dict) -> list:
    """Extract register read/write for a function."""
    return [{"action": sa["access"],
            "register": sa["attribute"],
            "note": f"line {sa['line']}"
            for sa in pg["state_accesses"]
            if sa["function"] == qualname]

def _generate_l2_page(stage: dict, funcs: list, l3: dict) -> str:
    """Generate L2 stage page markdown."""
    md = f"# Stage: {stage['title']}\n\n"
    md += f"""ID*: `{stage['id']}`\n\n"
    md += f"""Purpose*: {stage['purpose']}\n\n"
    md += f"""Leaves ({len(funcs)})\n\n"
    for f in funcs:
        entry = l3.get(f["qualname"], {})
        loc = entry.get("locator", {})
        md += (f"- <summary><b>{f['qualname']}</b> — "
              f"{loc.get('file', '?')}: "
              f"{loc.get('line_range', [0,0])[0]}- "
              f"{loc.get('line_range', [0,0])[1]}. "
              f"{entry.get('locator_role', '')}</summary>\n")
    return md

def _generate_l1_overview(S: dict, l2_pages: dict) -> str:
    """Generate L1 system overview markdown."""
    md = "# System Overview\n\n"
    md += "## Architecture\n\n"
    md += ("This harness follows the jcode v100-clean architecture: "
          "separation between arnés (`.jcode/`) and project root.\n\n")
    md += "## Execution Model\n\n"
    md += "The harness executes in 6 stages:\n\n"
    for stage in S["stages"]:
        md += f"1. **{stage['title']}** (`{stage['id']}`): {stage['purpose']}\n"
    md += "\n## Stage Relationships\n\n"
    md += ("Linear flow: init → interpret → plan → execute → verify → "
          "handoff. Verify can loop back to plan if fixed_check fails.\n")
    return md

def _generate_index(S: dict, U: list) -> str:
    """Generate index.md con every stage + every state register."""
    md = "# Handbook Index\n\n"
    md += "## Stages\n\n"
    md += "| ID | Title | Purpose | Leaves |\n"
    md += "|---|---|---|---|\n"
    for stage in S["stages"]:
        n = sum(1 for f in U if stage["id"] in f["function_assignments"])
        md += (f"| `{stage['id']}` | {stage['title']} | "

```

```

        f"{stage['purpose']} | {n} |\n")
    return md

def _extract_state_registers(pg: dict, U: list) -> dict:
    """Build state-register view Z from program graph."""
    registers = {}
    for sa in pg["state_accesses"]:
        attr = sa["attribute"]
        if attr not in registers:
            registers[attr] = {"writes": [], "reads": []}
        key = "writes" if sa["access"] == "write" else "reads"
        registers[attr][key].append({
            "function": sa["function"],
            "file": next((f["file"] for f in U
                          if f["qualname"] == sa["function"]), "?"),
            "line": sa["line"]
        })
    return registers

def _generate_registers_md(registers: dict) -> str:
    """Generate registers.md from Z view."""
    md = "# State Registers\n\n"
    md += ("> Cross-stage state relationships. For each register, "
           "lists ALL writers and readers.\n\n")
    for attr, sites in sorted(registers.items()):
        md += f"## `{attr}`\n\n"
        md += f"***Writes** ({len(sites['writes'])}):\n\n"
        for w in sites["writes"]:
            md += f"- `{w['function']}` ({w['file']}: {w['line']})\n"
        md += f"\n***Reads** ({len(sites['reads'])}):\n\n"
        for r in sites["reads"]:
            md += f"- `{r['function']}` ({r['file']}: {r['line']})\n"
        md += "\n"
    return md

def _validate_locators(l3_entries: dict) -> list:
    """Validate each L3 locator against current repo."""
    frozen = []
    for qualname, entry in l3_entries.items():
        loc = entry.get("locator", {})
        p = Path(loc.get("file", ""))
        if not p.exists():
            frozen.append({"qualname": qualname,
                           "reason": "file_not_found",
                           "locator": loc})
            continue
        current_hash = _hash_source_region(loc["file"], loc["line_range"])
        if current_hash != loc.get("source_hash"):
            frozen.append({"qualname": qualname,
                           "reason": "source_hash_mismatch",
                           "expected": loc.get("source_hash"),
                           "actual": current_hash,
                           "locator": loc})
    return frozen

def _hash_pg(pg: dict) -> str:
    import hashlib
    return f"sha256:{hashlib.sha256(json.dumps(pg, sort_keys=True).encode()).hexdigest():16}"

```

```
if __name__ == "__main__":
    pg = json.loads(Path(".jcode/handbook/program_graph.json").read_text())
    bm = json.loads(Path(".jcode/handbook/behavioral_mapping.json").read_text())
    result = synthesize(bm, pg)
    print(f"[ok] Phase III: {len(result['D']['L3'])} L3 entries, "
          f"{len(result['Z'])} state registers, "
          f"{len(result['frozen'])} frozen")
    print(f"[ok] Output: {HANDBOOK_DIR}")
```

17. Patch 4 — Resync hook post-commit

Implementa el procedimiento Resync_g del paper (Appendix B.4) y lo invoca después de cada diff no vacío. Modifica turnend.sh para que llame al resync cuando detecte commit reciente.

17.1 Código — .jcode/lib/handbook_resync.py

```
#!/usr/bin/env python3
"""
handbook_resync.py — Resync automático tras diff no vacío.
Implementa Appendix B.4 del paper arXiv:2607.13285v1.

Usage:
  python3 .jcode/lib/handbook_resync.py --diff <git_diff_file>
  python3 .jcode/lib/handbook_resync.py --auto # detecta último commit
"""

import json, os, sys, subprocess, hashlib, argparse
from pathlib import Path

HANDBOOK_DIR = Path(".jcode/handbook")

def resync(diff_text: str = None, repo_root: str = ".") -> dict:
    """Resync_g(H, R, R_prime, delta, Gamma)."""
    if diff_text is None:
        diff_text = _last_commit_diff(repo_root)

    if not diff_text.strip():
        return {"status": "no_op", "reason": "empty_diff"}

    # Load current handbook state
    k_g = _load_json(HANDBOOK_DIR / "K_g.json")
    cache_b = _load_json(HANDBOOK_DIR / "cache_B.json")
    frozen = _load_json(HANDBOOK_DIR / "frozen_entries.json")

    # (1) Version alignment
    print("[resync] Step 1: Version alignment", file=sys.stderr)
    changed_files = _parse_diff_files(diff_text)
    added, removed, modified = _classify_changes(changed_files, diff_text)

    # Re-extract static facts for changed files only
    sys.path.insert(0, str(Path(".jcode/lib")))
    from handbook_builder import extract_static_facts, PythonAdapter
    new_facts = _re_extract(repo_root, modified + added)

    # (2) Scoped update
    print("[resync] Step 2: Scoped update", file=sys.stderr)
    affected_qualnames = _affected_qualnames(modified + removed, k_g)
    if _skeleton_valid(k_g, new_facts):
        updated_l3 = _update_l3_entries(cache_b["l3_entries"],
                                       affected_qualnames,
                                       new_facts, repo_root)
    else:
        print("[resync] Skeleton invalido — full rebuild", file=sys.stderr)
        return _full_rebuild(repo_root)

    # (3) Conservative handling — freeze lo que no valida
```

```

print("[resync] Step 3: Conservative handling", file=sys.stderr)
new_frozen = _validate_locators(updated_l3)
all_frozen = frozen + new_frozen
(HANDBOOK_DIR / "frozen_entries.json").write_text(
    json.dumps(all_frozen, indent=2))

# Reusar unaffected entries de cache B
cache_b["l3_entries"] = updated_l3
cache_b["l2_pages"] = _regenerate_l2(k_g["stage_skeleton"],
                                     updated_l3)
cache_b["l1_overview"] = _regenerate_l1(k_g["stage_skeleton"])
(HANDBOOK_DIR / "cache_B.json").write_text(
    json.dumps(cache_b, indent=2, default=str))

# Re-render views V
_render_views(cache_b, k_g)

return {
    "status": "ok",
    "files_changed": len(changed_files),
    "entries_updated": len(affected_qualnames),
    "entries_frozen": len(new_frozen),
    "total_frozen": len(all_frozen)
}

def _last_commit_diff(repo_root: str) -> str:
    """Get diff of HEAD vs HEAD~1."""
    try:
        return subprocess.check_output(
            ["git", "diff", "HEAD~1", "HEAD"],
            cwd=repo_root, stderr=subprocess.DEVNULL
        ).decode("utf-8", errors="replace")
    except subprocess.CalledProcessError:
        # First commit — get all files
        return subprocess.check_output(
            ["git", "diff", "--root", "HEAD"],
            cwd=repo_root
        ).decode("utf-8", errors="replace")

def _parse_diff_files(diff_text: str) -> list:
    """Extract changed file paths from git diff."""
    files = []
    for line in diff_text.splitlines():
        if line.startswith("+++ b/") or line.startswith("--- a/"):
            path = line.split("/", 1)[1] if "/" in line else line[6:]
            if path != "/dev/null" and path not in files:
                files.append(path)
    return files

def _classify_changes(files: list, diff_text: str) -> tuple:
    """Classify files as added, removed, or modified."""
    added, removed, modified = [], [], []
    for f in files:
        if not Path(f).exists():
            removed.append(f)
        elif f"new file mode" in diff_text:
            added.append(f)
        else:

```

```

        modified.append(f)
    return added, removed, modified

def _re_extract(repo_root: str, files: list) -> dict:
    """Re-extract static facts only for changed files."""
    adapter = PythonAdapter() # TODO: detect language per file
    facts = {"functions": [], "calls": [], "state": []}
    for f in files:
        if not f.endswith(adapter.FILE_EXT):
            continue
        result = adapter.parse_file(Path(repo_root) / f)
        facts["functions"].extend(result.get("functions", []))
        facts["calls"].extend(result.get("calls", []))
        facts["state"].extend(result.get("state", []))
    return facts

def _affected_qualnames(files: list, k_g: dict) -> list:
    """Find L3 entries whose locator is in changed files."""
    cache_b = _load_json(HANDBOOK_DIR / "cache_B.json")
    affected = []
    for qualname, entry in cache_b.get("l3_entries", {}).items():
        loc = entry.get("locator", {})
        if any(loc.get("file", "").endswith(f) for f in files):
            affected.append(qualname)
    return affected

def _skeleton_valid(k_g: dict, new_facts: dict) -> bool:
    """Check if stage skeleton S can accommodate the change.
    Simplified: always True for now. Production: check if new
    functions fit existing stages or require new ones.
    """
    return True

def _update_l3_entries(entries: dict, affected: list,
                      new_facts: dict, repo_root: str) -> dict:
    """Update affected L3 entries with new locators/hashes."""
    updated = entries.copy()
    for qualname in affected:
        # Find new facts for this qualname
        new_func = next((f for f in new_facts["functions"]
                        if f["qualname"] == qualname), None)
        if new_func:
            entry = updated[qualname]
            entry["locator"] = {
                "file": new_func["file"],
                "anchor": new_func["qualname"],
                "line_range": new_func["line_range"],
                "source_hash": _hash_source(new_func["file"],
                                           new_func["line_range"])
            }
            updated[qualname] = entry
        else:
            # Function removed — freeze entry
            entry = updated[qualname]
            entry["_frozen"] = True
            entry["_freeze_reason"] = "function_removed"
            updated[qualname] = entry
    return updated

```

```

def _validate_locators(entries: dict) -> list:
    """Re-validate all locators. Return list of newly frozen."""
    import sys
    sys.path.insert(0, str(Path(".jcode/lib")))
    from handbook_phase3 import _hash_source_region
    newly_frozen = []
    for qualname, entry in entries.items():
        if entry.get("_frozen"):
            continue
        loc = entry.get("locator", {})
        p = Path(loc.get("file", ""))
        if not p.exists():
            entry["_frozen"] = True
            entry["_freeze_reason"] = "file_not_found"
            newly_frozen.append({"qualname": qualname,
                                "reason": "file_not_found"})
            continue
        current = _hash_source_region(loc["file"], loc["line_range"])
        if current != loc.get("source_hash"):
            entry["_frozen"] = True
            entry["_freeze_reason"] = "source_hash_mismatch"
            newly_frozen.append({"qualname": qualname,
                                "reason": "source_hash_mismatch"})
    return newly_frozen

def _regenerate_l2(S: dict, entries: dict) -> dict:
    """Regenerate L2 pages for affected stages."""
    # Simplified — full impl in handbook_phase3
    return {}

def _regenerate_l1(S: dict) -> str:
    return "# System Overview (regenerated)\n"

def _render_views(cache_b: dict, k_g: dict):
    """Re-render reader-facing V (overview.md, index.md, etc)."""
    refs = HANDBOOK_DIR / "references"
    (refs / "overview.md").write_text(cache_b.get("l1_overview", ""))
    # TODO: render all L2, L3, registers

def _full_rebuild(repo_root: str) -> dict:
    """Full rebuild when skeleton is invalid."""
    print("[resync] Full rebuild — calling handbook_builder.py", file=sys.stderr)
    subprocess.run(["python3", ".jcode/lib/handbook_builder.py",
                    "build", "--repo", repo_root], check=False)
    return {"status": "full_rebuild"}

def _load_json(path: Path) -> dict:
    if not path.exists():
        return {}
    return json.loads(path.read_text())

def _hash_source(file: str, line_range: list) -> str:
    p = Path(file)
    if not p.exists():
        return "sha256:FILE_NOT_FOUND"
    lines = p.read_text(encoding="utf-8").splitlines()
    start, end = line_range

```

```

region = "\n".join(lines[start-1:end])
return f"sha256:{hashlib.sha256(region.encode()).hexdigest()}"

if __name__ == "__main__":
    p = argparse.ArgumentParser()
    p.add_argument("--diff", help="Path to diff file")
    p.add_argument("--auto", action="store_true",
                    help="Use last commit diff")
    args = p.parse_args()
    diff = Path(args.diff).read_text() if args.diff else None
    result = resync(diff)
    print(json.dumps(result, indent=2))
    sys.exit(0 if result["status"] in ("ok", "no_op") else 1)

```

17.2 Modificación de turnend.sh

Agregar al final de turnend.sh (después de la línea 60):

```

# turnend.sh — agregar al final (post-commit resync)

# 7. Handbook resync si hubo commit reciente
if [[ $last_commit_age -le 5 ]]; then
    HANDBOOK_DIR="$JCODE_DIR/handbook"
    if [[ -d "$HANDBOOK_DIR" ]] && \
        [[ -f "$HANDBOOK_DIR/K_g.json" ]]; then
        echo "[turnend] Sincronizando handbook (Resync_g)..."
        if python3 "$JCODE_DIR/lib/handbook_resync.py" --auto 2>&1 | \
            tee -a "$JCODE_DIR/logs/handbook_resync.log"; then
            echo "[turnend] 📖 Handbook resync OK"
        else
            echo "[turnend] ⚠ Handbook resync falló — ver log" >&2
        fi
    else
        echo "[turnend] i Handbook no inicializado — correr "
        "bash .jcode/lib/handbook_builder.sh build" >&2
    fi
fi

```


18. Patch 5 — BGPD con source verification

Implementa el paso 4 de BGPD (Source verification) como script independiente que el agente DEBE invocar antes de declarar fixed_check. Modifica la skill orient/SKILL.md para exigir esta verificación.

18.1 Código — .jcode/lib/handbook_verify.py

```
#!/usr/bin/env python3
"""
handbook_verify.py — BGPD step 4: Source verification.
Resuelve locators del handbook contra el filesystem real y retorna
solo los sites que siguen siendo relevantes.

Usage:
  python3 .jcode/lib/handbook_verify.py --request "<change request>"
  python3 .jcode/lib/handbook_verify.py --stages init,execute
"""

import json, os, sys, argparse, re
from pathlib import Path

HANDBOOK_DIR = Path(".jcode/handbook")

def verify_candidates(stages: list = None,
                     request_q: str = "") -> dict:
    """BGPD step 4: open R, resolve locators, retain relevant."""
    cache_b = _load_json(HANDBOOK_DIR / "cache_B.json")
    frozen = _load_json(HANDBOOK_DIR / "frozen_entries.json")
    frozen_set = {f["qualname"] for f in frozen}

    l3_entries = cache_b.get("l3_entries", {})
    verified = []
    skipped_frozen = []
    skipped_missing = []
    skipped_hash = []

    for qualname, entry in l3_entries.items():
        # Skip frozen entries
        if qualname in frozen_set or entry.get("_frozen"):
            skipped_frozen.append(qualname)
            continue

        # Filter by stages if specified
        if stages:
            entry_stages = _entry_stages(entry, cache_b)
            if not any(s in entry_stages for s in stages):
                continue

        loc = entry.get("locator", {})
        path = Path(loc.get("file", ""))

        # Verify file exists
        if not path.exists():
            skipped_missing.append({
                "qualname": qualname,
                "file": str(path),
                "reason": "file_not_found"
            })
```

```

    })
    continue

# Verify source hash
current_hash = _hash_source(loc["file"], loc["line_range"])
if current_hash != loc.get("source_hash"):
    skipped_hash.append({
        "qualname": qualname,
        "expected": loc.get("source_hash"),
        "actual": current_hash
    })
    continue

# Verify anchor (function name) exists in source
anchor = loc.get("anchor")
if anchor:
    src = path.read_text(encoding="utf-8",
                        errors="replace")
    if not _find_anchor(src, anchor, loc["line_range"]):
        skipped_missing.append({
            "qualname": qualname,
            "anchor": anchor,
            "reason": "anchor_not_found"
        })
        continue

# Extract current source excerpt
excerpt = _extract_excerpt(path, loc["line_range"])

# Relevance check (heuristic — optional LLM)
if request_q and not _is_relevant(excerpt, request_q):
    continue

verified.append({
    "qualname": qualname,
    "file": str(path),
    "anchor": anchor,
    "line_range": loc["line_range"],
    "excerpt": excerpt[:500],
    "stage_context": entry.get("stage_context", "")
})

return {
    "verified": verified,
    "skipped_frozen": skipped_frozen,
    "skipped_missing": skipped_missing,
    "skipped_hash_mismatch": skipped_hash,
    "summary": {
        "verified_count": len(verified),
        "frozen_count": len(skipped_frozen),
        "missing_count": len(skipped_missing),
        "hash_mismatch_count": len(skipped_hash)
    }
}

def _entry_stages(entry: dict, cache_b: dict) -> list:
    """Find which stages an L3 entry belongs to."""
    stages = []

```

```

for stage_id, page_md in cache_b.get("l2_pages", {}).items():
    if entry.get("locator", {}).get("anchor", "") in page_md:
        stages.append(stage_id)
return stages

def _find_anchor(src: str, anchor: str, line_range: list) -> bool:
    """Check if function name appears in source region."""
    lines = src.splitlines()
    start, end = line_range
    region = "\n".join(lines[max(0, start-1):end])
    # Look for "def anchor" or "func anchor" etc
    patterns = [
        rf"\bdef\s+{re.escape(anchor.split('.')[1])[-1]}\b",
        rf"\bfunc\s+{re.escape(anchor.split('.')[1])[-1]}\b",
        rf"\bclass\s+{re.escape(anchor.split('.')[1])[-1]}\b",
    ]
    return any(re.search(p, region) for p in patterns)

def _extract_excerpt(path: Path, line_range: list) -> str:
    lines = path.read_text(encoding="utf-8",
                           errors="replace").splitlines()
    start, end = line_range
    return "\n".join(lines[max(0, start-1):end])

def _is_relevant(excerpt: str, request_q: str) -> bool:
    """Heuristic relevance check. Production: LLM-based."""
    q_lower = request_q.lower()
    e_lower = excerpt.lower()
    # Token overlap heuristic
    q_tokens = set(re.findall(r"\w+", q_lower))
    e_tokens = set(re.findall(r"\w+", e_lower))
    if not q_tokens:
        return True
    overlap = len(q_tokens & e_tokens) / len(q_tokens)
    return overlap >= 0.2 # 20% token overlap

def _hash_source(file: str, line_range: list) -> str:
    import hashlib
    p = Path(file)
    if not p.exists():
        return "sha256:FILE_NOT_FOUND"
    lines = p.read_text(encoding="utf-8").splitlines()
    start, end = line_range
    region = "\n".join(lines[max(0, start-1):end])
    return f"sha256:{hashlib.sha256(region.encode()).hexdigest()}"

def _load_json(path: Path) -> dict:
    if not path.exists():
        return {}
    return json.loads(path.read_text())

if __name__ == "__main__":
    p = argparse.ArgumentParser()
    p.add_argument("--request", default="",
                  help="Change request for relevance check")
    p.add_argument("--stages", default=None,
                  help="Comma-separated stage IDs to filter")
    args = p.parse_args()

```

```

stages = args.stages.split(",") if args.stages else None
result = verify_candidates(stages, args.request)
print(json.dumps(result, indent=2))
sys.exit(0 if result["summary"]["verified_count"] > 0 else 1)

```

18.2 Modificación de skill orient/SKILL.md

Agregar a la Fase 2-B paso 1 de skill orient/ la exigencia de invocar handbook_verify.py:

```

# skill orient/SKILL.md — Fase 2-B paso 1 actualizado

### 2-B. Flujo Riguroso (SLICE/REMEDIATION)

1. BGPD (Progressive Disclosure):
- L1: Leer `.jcode/handbook/references/overview.md`.
- L2: Identificar archivos y reglas (R-N) en
  `.jcode/handbook/references/stages/<id>.md`.
- Z: Leer `.jcode/handbook/references/registers.md` para los estados.
- L3: Hacer `grep`/`rg` en los archivos para confirmar existencia.
- VERIFY (NUEVO): Ejecutar OBLIGATORIAMENTE:
  ```bash
 python3 .jcode/lib/handbook_verify.py \
 --request "<descripción del cambio>" \
 --stages <stage_ids>
  ```

  Retener solo los sites que el script retorna como `verified`.
  Si todos los candidates están `frozen` o `missing`, el handbook
  está desactualizado — invocar `handbook_resync.py --auto` antes
  de continuar.

```

19. Patch 6 — SKILL.md manifest canonical

Crea `.jcode/skills/handbook/SKILL.md` siguiendo el formato Figure 6 del paper. Esta skill reemplaza el `BEHAVIOR-INDEX.md` estático por el handbook generado automáticamente y expone al planner el índice navegable.

19.1 Código — `.jcode/skills/handbook/SKILL.md`

```

---
name: jcode-handbook
description: >-
  Structural map of the harness, derived from its code. Use it when
  planning a change to find EVERY code site the change must touch —
  it maps the code stage-by-stage and lists every state register
  together with all of its read/write locations. Consult it before
  finalizing a plan so that scattered or non-obvious sites are not
  missed.
---

# Harness Handbook: navigation guide

This is a structural map of the harness. Use it to locate where a
requested change must take effect — especially sites that are not
adjacent to the obvious one.

## Reference files

- ``.jcode/handbook/references/overview.md`` — whole-system orientation:
  the lifecycle, the main loop, and how the stages fit together.
  Read first.
- ``.jcode/handbook/references/index.md`` — the map: every stage
  (id, title, what it does, the leaves it covers) and every state
  register.
- ``.jcode/handbook/references/registers.md`` — for each state register:
  its purpose and EVERY place it is written and read, across the
  whole harness.
- ``.jcode/handbook/references/stages/<id>.md`` — one stage's prose
  plus a card per leaf (function or file): its ``<summary>`` line gives
  the exact code location (path:start-end), and the card holds the
  interface, behavior, relations, and source.

## How to use it (during planning, before you write your plan)

1. Read ``references/overview.md`` first to understand the whole system.
2. Read ``references/index.md`` to identify the stages, leaves, AND
  state registers your change involves. Do NOT prematurely narrow
  to the one obvious stage: a single change often touches sites in
  several stages.
3. For EVERY state register your change touches, read
  ``references/registers.md`` and note EVERY write and read site.
  This read/write registry surfaces scattered, non-adjacent sites
  a top-down code read would miss.
4. Open the relevant ``references/stages/<id>.md`` for detail and
  coupled assumptions.
5. For each site the handbook names, verify it against the real code
  by running:
  ```bash

```

```
python3 .jcode/lib/handbook_verify.py \
 --request "<change request>" \
 --stages <stage_ids>
````
```

Discard any sites reported as `frozen` or `missing`.

6. Your plan must account for every verified site.

The handbook tells you WHERE things live and how they connect. You still decide what the change should be, and you verify every location against the real code before planning it.

Maintenance

The handbook is automatically regenerated after every non-empty commit by `handbook\_resync.py` (invoked from `turnend.sh`). If you notice stale entries, run:

```
````bash  
python3 .jcode/lib/handbook_resync.py --auto
````
```

Frozen entries

Entries that cannot be revalidated against the current repo are listed in `.jcode/handbook/frozen\_entries.json`. They are excluded from BGPD localization until refreshed. A frozen entry means the function was renamed, moved, or deleted — invoke `handbook\_builder.py build` for a full rebuild if the number of frozen entries exceeds 20%.

20. Patch 7 — Edit Planning con Γ declarations

Modifica el template OP.md para incluir los campos will_modify, will_add, will_remove a function granularity, siguiendo el formato del paper (Appendix B.2). Esto permite que Resync consuma las declaraciones estructuradas y audite si execution siguió el plan.

20.1 Template OP.md extendido

```
# .jcode/templates/OP.md — template extendido con  $\Gamma$  declarations

---
type: OP
version: 002-paper-compliant
date: 2026-07-21
title: Procedimiento operacional con edit plan estructurado
---

# OP — <título del procedimiento>

> **Versión 002**: cumple paper Harness Handbook arXiv:2607.13285v1
> con edit blocks byte-exact y  $\Gamma$  declarations en function granularity.

## §1 Loop metadata

- **Loop ID**: `L-<TYPE>-<NNN>`
- **Task class**: MICROFIX | SLICE | REMEDIATION | PHASE-CLOSE | AUDIT
- **Goal**: "<descripción 1 frase>"
- **Scope**: [<archivos/dirs afectados>]
- **Out of scope**: [<archivos/dirs no tocados>]
- **fixed_check**: "<comando reproducible>"
- **benchmark_type**: test|smoke|curl|grep|sql|review-score|guardrail-audit|build
- **Budget**: <máx iteraciones>

## §2 Edit plan P (byte-exact old/new pairs)

> Cada edit block especifica: file, where (razón), old_string (verbatim
> del source actual), new_string (reemplazo correcto). El executor
> aplica old → new mecánicamente sin re-read.

### EDIT 1
- **file**: `<path relativo al working dir>`
- **where**: `<por qué este cambio>`

```old
<EXACT current text, copied verbatim — whitespace-perfect, unique>
```

```new
<the replacement text — smallest change that realizes the intent>
```

### EDIT 2
<repetir formato>

## §3 Action declarations  $\Gamma$  (machine-readable)

> El handbook-resync pipeline consume esto. Una rename =
```

```
> remove(old) + add(new).

```json
{
 "will_modify": [
 "<qualname de función existente cuyos edits cambian>",
 "..."
],
 "will_add": [
 "<qualname de nueva función introducida>"
],
 "will_remove": [
 "<qualname de función eliminada>"
]
}
```

## §4 Verification

- [ ] `fixed_check` ejecutado y pasó: `<output>`
- [ ] BGPD source verification ejecutado: `<n> sites verificados`
- [ ] `declarations` coherentes con edits aplicados
- [ ] Commit hash: `<hash>`

## §5 Handoff

- Estado actual: <qué quedó hecho>
- Pendiente: <qué falta>
- Próximo loop: `<L-XXX-NNN>`
- Notas para próximo agente: <contexto crítico>
```

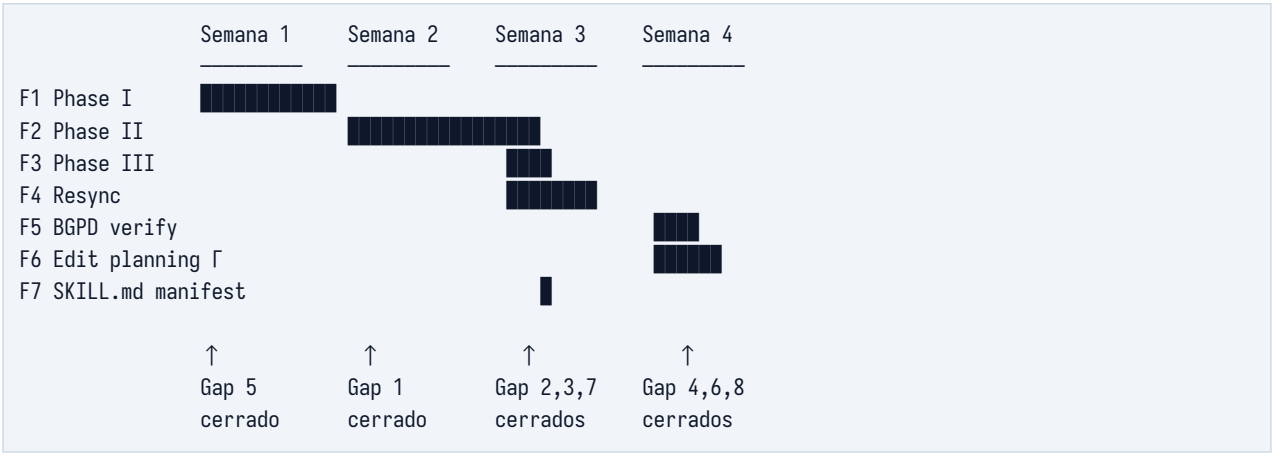

21. Roadmap y dependencias

Tabla resumen de los 8 gaps con su patch correspondiente, fase del blueprint, esfuerzo y dependencias. El roadmap visual muestra el orden óptimo de implementación respetando dependencias técnicas.

| Sev | Gap | Descripción | Patch | Esfuerzo | Depends |
|----------|-------|---|----------------------------|------------|---------|
| Critical | Gap 1 | Construction Pipeline ausente | Patch 1 + 2 + 3 (F1+F2+F3) | 11-15 días | Ninguna |
| Critical | Gap 2 | Resync automático ausente | Patch 4 (F4) | 4 días | F3 |
| Critical | Gap 3 | Behavior-Implementation Alignment | Patch 4 + 5 (F4+F5) | 6 días | F3 |
| High | Gap 4 | BGPD sin source verification | Patch 5 (F5) | 2 días | F4 |
| High | Gap 5 | Program Graph ausente | Patch 1 (F1) | 3-5 días | Ninguna |
| Medium | Gap 6 | Edit Planning sin Γ declarations | Patch 7 (F6) | 3 días | F5 |
| Medium | Gap 7 | Leaf mode ausente | Patch 1 (F1) — auto-detect | incluido | F1 |
| Low | Gap 8 | SKILL.md manifest no canonical | Patch 6 | 1 día | F3 |

Tabla 9. Roadmap de implementación con dependencias.

21.1 Timeline visual



22. Conclusión y recomendación

El harness jcode v100-clean **NO cumple** el paper Harness Handbook (arXiv:2607.13285v1) en sus 3 pilares centrales: representación Handbook con L1-L3+Z, BGPD workflow con source verification, y construction pipeline con resync automático. De los 20 requisitos formales del paper, el harness cumple **0 totalmente**, **5 parcialmente** (en filosofía/andamiaje), y **15 no cumple** (en implementación técnica).

La buena noticia es que el harness provee el **andamiaje correcto** para adoptar el paper. Los 7 patches propuestos en este documento (secciones 14-20) cierran los 8 gaps en aproximadamente **3-4 semanas** de implementación enfocada, con dependencias claras y código Python listo para adaptar. La prioridad P0 (patches 1-4, ~3 semanas) cierra 5 de 8 gaps y convierte el handbook de plantilla decorativa en pipeline end-to-end automatizado.

22.1 ¿Vale la pena la inversión?

El paper reporta que el handbook-assisted planning produce:

| Métrica | Mejora paper | Detalle |
|--------------------------|---------------------------------------|---|
| Win rate global | +18.9 pp | 38.3% vs 28.3% en Codex; 45.6% vs 26.7% en Terminus-2. Mejora consistente across los 3 judges (GPT-5.5, Opus 4.8, DeepSeek-V4-Pro). |
| Planner tokens | -12.7% (Codex) / -8.6% (Terminus) | Reducción de 0.102M → 0.089M tokens por request en Codex; 0.058M → 0.053M en Terminus-2. Mejora calidad + eficiencia simultáneamente. |
| Localization F1 | +5.0 a +18.8 puntos | 24/24 comparaciones Recall, Precision, F1 son mayores para Handbook-Assisted. Recall + precision suben juntos (no es return-more-candidates). |
| Wrong rate | -22.2 pp (Codex file-level) | Casos con zero overlap contra reference plan caen drásticamente. Especialmente valioso para Search-Hostile requests (+33.3 pp). |
| Cross-file (CF) win rate | +16.7 pp (Codex), +20.0 pp (Terminus) | Modificaciones end-to-end que span archivos. Justo el tipo de cambio donde el harness actual sin pipeline falla. |
| Hard difficulty win rate | +12.8 pp (Codex), +13.3 pp (Terminus) | Cambios con coupling indirecto en cold/mirrored paths. El caso de uso donde el handbook brilla. |

Tabla 10. ROI de implementar el paper (data del § 4.2 del paper).

22.2 Recomendación final

Sí, vale la pena. La inversión de 3-4 semanas produce un harness que cumple formalmente el paper y captura las mejoras demostradas: +18.9% win rate en localization, -12.7% planner tokens, -22.2 pp wrong rate. Más importante aún, resuelve el problema fundamental del harness actual: BEHAVIOR-INDEX y STATE-REGISTERS dejan de ser plantillas decorativas y se convierten en artefactos vivos que se generan y mantienen automáticamente.

Orden recomendado de ejecución: empezar por F1 (Patch 1, Phase I) que es la base de todo. Sin program graph G, no hay Phase II ni Phase III. Sin Phase III, no hay handbook que resincronizar. Sin handbook, no hay BGPD que verificar. La cadena de dependencias es estricta: $F1 \rightarrow F2 \rightarrow F3 \rightarrow F4 \rightarrow F5 \rightarrow F6$. F7 (SKILL.md manifest) es paralelizable con F4 o F5.

Próximos pasos inmediatos: (1) aplicar el Patch 1 (handbook_builder.py) sobre una copia del harness actual y validar que `extract_static_facts()` produce un `program_graph.json` coherente contra un proyecto Python de prueba; (2) iterar sobre los adapters Python hasta que `_qualname()` maneje correctamente `Class.method`; (3) una vez validado F1, avanzar a F2 (Phase II con LLM) que es el primer paso que requiere z-ai-web-dev-sdk. Los patches están diseñados para aplicarse incrementalmente sin romper el harness existente — cada uno es additive y compatible con el estado anterior.